Problem #1: Installing Python and Python packages

Throughout this course, you'll regularly use Python to perform calculations, analyze data, and make figures to support conclusions you draw. In order to do this, you'll need to install a minimum set of packages if you don't already have them. First, make sure you have a working Python installation. My preferred method is using miniconda with Python version 3.10 which is a lightweight Python package manager. You are welcome to use other package managers if you have a preference!

Once you have Python installed, there are a few packages that will prove to be very useful! Please install at a minimum the Python packages listed below. Many will have a conda installation available but it's not gauranteed. You can check to see whether a package is available using the conda search function.

numpy	matplotlib	astropy	jupyterlab	astroquery

Problem #2: Accessing astronomical data from public databases

One of the most powerful datasets in astronomy is the latest (third) release from the Gaia satellite. This is partially the case because the data is public and thus free to use for any astronomical research. One of the easiest ways to access Gaia data is through the Gaia ESA Archive. In the top of the archive webpage, there are tabs that you can use to perform different searches. Click on the "SEARCH" tab to access the standard search form.

Problem #2a: Searching for individual sources

Some stars are bright enough that they have been known of since ancient times because we can see them with our eyes. Because of this, they have names like Beta Pictoris instead of phone numbers like most modern stars (e.g. Gaia DR3 4373465352415301632). You can search for both kinds of stars individually by name in the "SINGLE OBJECT" tab. For both Beta Pictorus and Gaia DR3 4373465352415301632, gather the mean values for the parallax, apparent brightness in Gaia's G filter, and the mean effective temperature and report them as your answer.

Problem #2b: Using bolometric corrections

Using the Bolometric correction equation $BC_{\rm G} = M_{\rm bolometric} - M_{\rm G}$ and the following bolometric correction values for Beta Pictoris and Gaia DR3 4373465352415301632, calculate the apparent bolometric magnitude for each star. Based on the effective temperatures, why is Beta Pic's bolometric correction larger? What does this tell us about the Gaia G filter?

Beta Pictoris:	$BC_G = 0.0172$
Gaia DR3 4373465352415301632:	$BC_G = 0.0035$

Problem #2c: Calculating bolometric luminosities

In order to calculate the bolometric luminosity, we need to first calculate the *absolute magnitude* using the distance modulus : $M_{\rm abs} = M_{\rm app} - 5\log_{10}(d/10{\rm pc})$. This requires the distance, which can be estimated from the parallax.

First estimate the absolute bolometric magnitude of each star, using the estimated distance. Then, using the absolute bolometric magnitude of the Sun, calculate the bolometric luminosity of both Beta Pictoris and Gaia DR3 4373465352415301632.

Problem #2d: Using the Stefan-Boltzmann Equation to infer stellar radii

Now that we have the bolometric luminosity of each star, we can use the measured effective temperature and the Stefan-Bolztmann Equation to infer the radius of each star. Calculate the radius of both Beta Pictoris and Gaia DR3 4373465352415301632. How do they compare to the Sun's radius?

Problem #2e: Using the Main-Sequence Mass–Luminosity relation to infer stellar masses

The Mass–Luminosity relation allows us to estimate the masses of main-sequence stars by comparing their bolometric luminosities to the bolometric luminosity of the Sun. Use the M–L relation to estimate the mass of both Beta Pictoris and Gaia DR3 4373465352415301632. Models using high-resolution spectra of both stars have been used to perform fits to the masses of each star which are listed below. How do the M–L relation estimates compare? Compute the percentage difference between the spectral fits and the M–L estimates.

Beta Pictoris:	$M = 1.75\,{\rm M}_\odot$
Gaia DR3 4373465352415301632:	$M = 0.93\,{\rm M}_\odot$

Problem #3: Downloading astronomical data from ADQL

If we are instead interested in accessing data from a *population* of stars, it is not efficient to use individual searches for each star. The Astronomical Data Query Language (ADQL) is a language similar to SQL that allows users to search large astronical databases to select subpopulations of interest. Gaia DR3 contains almost *2 billion* sources with photometric measurements. This means that we should not try to download the entire dataset (it will 100% crash your laptop!).

Problem #3a: ADQL with `astroquery`

The astroquery Python package provides an interface to the Gaia database so that you can query data directly using ADQL. This interface is extremely powerful and thus has a *large* amount of documentation. For now, let's use the following astroquery syntax to select a subset of 1000 stars with well-measured parallaxes:

```
from astroquery.gaia import Gaia
```

```
gaiadr3_table = Gaia.load_table('gaiadr3.gaia_source')
```

```
Retrieving table 'gaiadr3.gaia_source'
```

```
job = Gaia.launch_job("select top 5000 "
                      "source_id, phot_g_mean_mag, "
                      "phot_bp_mean_mag, phot_rp_mean_mag, parallax "
                      "from gaiadr3.gaia_source where parallax_over_error > 20 and parallax > 0.2")
r = job.get_results()
```

You should supply the same query by opening a jupyterlab environment and repeating the same import and query as above. Try printing the results table `r`, which shows each star as a separate row. How does the requirement for parallax > 0.2 mas influence the distance of the stars in our table?

Notice that `astroquery` returns a table; this is an `astropy` table which allows you to store *unit*s in addition to values for each star. Fields with units are called quantities and can be manipulated with the `astropy.units` class.

Problem #3b: Plotting a color-magnitude diagram (CMD)

Using the `matplotlib` package, plot a scatter plot that shows the color-magnitude diagram which contains the apparent G magnitude on the y axis and the BP - RP magnitude on the x axis for each source. Be sure to label each axis!

If you are unfamiliar with `matplotlib`, the best way to learn is often a google search with "matplotlib" appended. For example, for this figure, you could search: "matplotlib scatter plot" and "matplotlib xlabel". You may also find the pyplot tutorial helpful!

Problem #3c: Converting from parallax to distance with equivalencies in `astropy.units`

Now that we have a column with parallax, we can calculate the distance assuming that all motion on the sky for each source is due to parallax. The easiest way to do this is through equivalencies. In order to use equivalencies, you'll need to import the units class as

```
import astropy.units as u
```

You should convert the parallax to distance and store the distance values in a new column in your `r` results table as

```
r['distance'] = r['parallax'].to(u.pc, equivalencies=u.parallax())
```

Problem #3d: Creating an absolute CMD

Using the distance column and the apparent magnitude columns, create *absolute* G, BP, and RP columns in your `r` results table. You can do this using the distance modulus calculation you did above in question 2. Now plot the color-magnitude diagram, but with the *absolute* G and BP-RP values. What do you notice has changed? Finally, remake the figure, but change the y-axis to range from high G to low G (upside down). This is an absolute color-magnitude diagram which contains stars at different evolutionary phases.